SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CSA47 – DEEP LEARNING

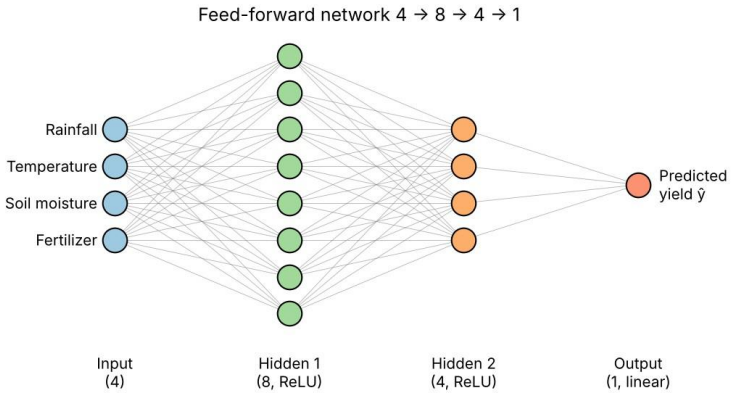
ASSIGNMENT – 2
CO2 – Deep Feed-Forward and Convolutional Network Design

Name: P.Prasanna Sree
Reg No:192324309
Slot B

TASK 1 – Deep Feed-Forward Network Architecture

a) Network design

The inputs are rainfall, temperature, soil moisture and fertilizer quantity, so the input layer has **4 neurons**. The two hidden layers have **8** and **4** neurons. Crop yield is a single continuous value (regression), so the output layer has **1 neuron** with a linear (identity) activation. ReLU is used in the hidden layers. The architecture is **4 → 8 → 4 → 1**.



Layer	Neurons	Activation	Role
Input	4	–	Receives normalized rainfall, temperature, soil moisture, fertilizer
Hidden 1	8	ReLU	Learns combinations of the four inputs
Hidden 2	4	ReLU	Learns higher-level combinations of hidden-1 features
Output	1	Linear	Predicted crop yield ŷ

b) Dimensions of weight matrices and bias vectors:

With the column-vector convention, the weight matrix of a layer has shape (neurons in current layer × neurons in previous layer), and the bias vector has one entry per neuron in the current layer.

Parameter	Dimension	Number of values
W [1] (input → hidden 1)	8 × 4	32
b [1]	8 × 1	8
W [2] (hidden 1 → hidden 2)	4 × 8	32
b [2]	4 × 1	4
W [3] (hidden 2 → output)	1 × 4	4
b [3]	1 × 1	1
Total trainable parameters		81

c) Forward propagation (mathematical representation):

Let x = [rainfall, temperature, soil moisture, fertilizer]T (4 × 1). The inputs flow layer by layer:

z [1] = W[1]x + b[1], a[1] = ReLU(z[1]) (8 × 1)

z [2] = W[2]a [1] + b[2], a[2] = ReLU(z[2]) (4 × 1)

$\hat{y}$ = z[3] = W[3]a [2] + b[3] (1 × 1)

Each neuron computes a weighted sum of the previous layer's outputs plus a bias, and the activation function adds non-linearity. Without non-linear activations, the three layers would collapse into one linear model. The final value $\hat{y}$ is the crop-yield prediction. For a batch of m records, X is 4 × m and the same equations produce a 1 × m vector of predictions.

TASK 2 – Forward Propagation and Activation Functions

a) Weighted sum Inputs

x = (0.6, 0.8, 0.5), weights w = (0.4, 0.3, 0.2), bias b = 0.1.

$z = w_1x_1 + w_2x_2 + w_3x_3 + b = (0.4)(0.6) + (0.3)(0.8) + (0.2)(0.5) + 0.1$

z = 0.24 + 0.24 + 0.10 + 0.10 = 0.68

b) ReLU and interpretation

ReLU(z) = max(0, z) = max(0, 0.68) = 0.68

Because z is positive, ReLU passes it unchanged, so the neuron is active and sends 0.68 to the next layer. If z had been negative, the output would be 0 (neuron inactive). This shows that ReLU switches a neuron on only when the weighted evidence is positive.

TASK 3 – Gradient-Based Learning and Loss Calculation

a) Need for a loss function and suitable choice

A loss function converts the difference between prediction and actual value into a single number that measures how wrong the network is. Learning needs this number because gradient-based optimization has to minimize something differentiable; without a loss there is no direction for improving the weights. For regression (continuous yield), Mean Squared Error (MSE) is suitable: it is smooth and differentiable, and it penalizes large errors more strongly. Mean Absolute Error or Huber loss are alternatives when outliers are present.

b) Squared error loss

$L = (y - \hat{y})^2 = (50 - 42)^2 = 8^2 = 64$

c)Interpretation

The prediction is 8 units below the actual yield (under-prediction), giving a loss of 64 (RMSE = 8 units). A loss of 0 would mean a perfect prediction; the larger the loss, the poorer the prediction. A loss of 64 is large in this context, indicating that the weights still need substantial adjustment. The gradient with respect to the prediction is $\partial L/\partial \hat{y} = -2(y - \hat{y}) = -16$, a negative value which indicates that increasing $\hat{y}$ would reduce the loss.

d) Gradient-based learning

1) Forward pass: compute $\hat{y}$ and the loss L.

2) Backward pass: compute the gradient $\partial L/\partial w$ for every weight and bias; this gradient tells how the loss changes if the parameter changes slightly.

3) Update each parameter by moving it opposite to its gradient: $w \leftarrow w - \eta\, \partial L/\partial w$, where η is the learning rate.

4) Repeat over many iterations (mini-batches and epochs) until the loss stops decreasing. Over time the parameters move towards values that minimize the loss.

TASK 4 – Gradient Descent and Weight Update

a) Updated weight

$W_{new} = w - \eta(\partial L/\partial w) = 0.5 - (0.1)(0.8) = 0.5 - 0.08 = 0.42$

b) Why the update is opposite to the gradient

The gradient points in the direction in which the loss increases most rapidly. To reduce the loss, the weight must move in the opposite direction (steepest descent). Here the gradient is positive, so increasing the weight would increase the loss; the weight is therefore decreased from 0.5 to 0.42.

c) Effect of very small or very large learning rate

• Extremely small η (e.g., 0.0001): $w = 0.5 - 0.0001 \times 0.8 = 0.49992$. The updates are tiny, training is very slow, needs many epochs, and may stall on plateaus or in poor local regions.
• Extremely large η (e.g., 5): $w = 0.5 - 5 \times 0.8 = -3.5$. The step overshoots the minimum, the loss oscillates or increases, and training may diverge.

• A moderate η (with decay) gives steady and stable convergence.

TASK 5 – Back-Propagation Through Multiple Layers

a) Contribution of weights to the error After the forward pass computes ŷ and the loss, back-propagation sends the error backwards from the output layer through hidden layer 2, hidden layer 1 and finally to the input layer. At every layer it calculates the error signal δ and the gradient ∂L/∂W, which measures how much each weight contributed to the final error. Weights that strongly influenced a large error receive large gradients; those with little influence receive small gradients.

b) Role of the chain rule The loss depends on the weights through a chain of composed functions (loss ← output ← hidden 2 ← hidden 1 ← weights).

The chain rule multiplies the local derivatives along this path. For example, for a first-layer weight:

∂L/∂W[1] = (∂L/∂ŷ)(∂ŷ/∂a[2])(∂a[2]/∂z[2])(∂z[2]/∂a[1])(∂a[1]/∂z[1])(∂z[1]/∂W[1])

Reusing the already computed δ of later layers makes this efficient: δ[3] = ∂L/∂ŷ, δ[2] = (W[3]) T δ [3] ⊙ f'(z[2]), δ [1] = (W[2]) T δ [2] ⊙ f'(z[1]).

d) Using gradients to update weights

Each weight and bias is updated by W[l] ← W[l] − η ∂L/∂W[l] and b[l] ← b[l] − η ∂L/∂b[l]. Since ∂L/∂W[l] = δ [l](a[l−1]) T , the updated network produces a prediction closer to the actual yield in the next iteration.

TASK 6 – Activation Functions and Vanishing/Exploding Gradients

a) **How activation functions cause vanishing gradients**

During back-propagation the gradient is multiplied by the derivative of the activation at every layer. The sigmoid derivative has a maximum value of only 0.25 and approaches 0 when the neuron saturates (|z| large). With n layers the gradient shrinks roughly like 0.25n (for 5 layers ≈ 0.001). Early layers therefore receive almost no gradient and learn very slowly. Conversely, if weights and derivatives are repeatedly greater than 1, the gradient grows exponentially (exploding gradients), which gives unstable updates or NaN values.

b) Comparison of sigmoid, tanh and ReLU

Property	Sigmoid	Tanh	ReLU
Formula	$\frac{1}{1+e^{-z}}$	$\frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	$\max(0,z)$
Output range	(0, 1)	(−1, 1)	[0, ∞)
Maximum derivative	0.25	1	1 (for $z > 0$)
Zero-centred	No	Yes	No

Saturation / Vanishing gradient	Severe	Present, less severe	None for $z > 0$
Computation	Exponential (costly)	Exponential (costly)	Very cheap
Main issue	Vanishing gradient, slow learning	Vanishing gradient	Dying ReLU ($z \leq 0$ gives zero gradient)
Typical Issue	Output for binary probability	Hidden layers in shallow nets/RNNs	Default for hidden layers

c) Effect on deep-network training

The choice of activation controls how well gradients travel to early layers. ReLU keeps a gradient of 1 for active neurons, allowing faster and more stable convergence in deep networks; sigmoid causes slow learning in the early layers. Problems with ReLU (dead neurons) can be reduced with Leaky ReLU/ELU, and gradient problems in general with He initialization, batch normalization, residual connections and gradient clipping (for exploding gradients). For the crop-analysis network, replacing sigmoid by ReLU in hidden layers is the first recommended change.

TASK 7 – Back-Propagation and Optimization Strategy

a) Effect of learning rate on optimization The learning rate η sets the step size.

A small η gives a smooth but slow decrease of loss (Experiment

1). A large η gives fast initial progress but overshoots the minimum, so the loss fluctuates strongly or diverges (Experiment 2).

b) Backpropagation and gradient descent in one iteration

• Forward pass: a mini-batch is passed through the network to obtain predictions and the loss.

• Backpropagation: the chain rule computes ∂L/∂W and ∂L/∂b for all layers.

• Gradient descent: the parameters are updated using W ← W − η ∂L/∂W.

• Back-propagation therefore computes the gradients; gradient descent uses them to change the weights.

b) Suggested improvements

Experiment	Observation	Suggested changes
1	Loss decreases slowly	Increase learning rate; use Adam or momentum; normalize inputs; He initialization; batch normalization; ReLU instead of sigmoid; check for vanishing gradients
2	Loss decreases quickly but fluctuates	Reduce learning rate; use a decay schedule; increase mini-batch size; use momentum/Adam; gradient clipping; early stopping on the best validation loss

c) Why the loss curve should be monitored

Plotting training and validation loss shows whether the model is converging, whether the learning rate is suitable (slow/fluctuating/diverging curve), and whether overfitting starts (validation loss rises while training loss falls). It supports decisions on learning-rate changes, early stopping and when to end training (see Graph 2).

TASK 8 – Regularization and Overfitting Control

a) Likely problem and evidence

The likely problem is overfitting (high variance). Evidence: training accuracy is 99% while validation accuracy is only 78%, a generalization gap of 21 percentage points. The network has memorized the training examples, including noise, instead of learning patterns that generalize to unseen data.

b) L1 and L2 regularization

• L1 adds $\lambda\Sigma|w|$ to the loss. It pushes many weights exactly to zero, giving sparse models and a form of feature selection.

• L2 (weight decay) adds $\lambda\Sigma w^2$. It shrinks all weights smoothly towards zero without making them exactly zero, preventing any single weight from becoming very large and giving a smoother function.

• Both limit the effective complexity of the model, reducing variance.

c) Regularized loss with L2 penalty

$$L_{reg} = (1/N)\ \Sigma\ (y_i - \hat{y}_i)^2 + \lambda\ \Sigma_l\ \|W[l]\|^2$$

(For the crop-category classification, the first term is replaced by cross-entropy loss.) The regularization parameter λ controls the strength of the penalty: $\lambda = 0$ means no regularization (overfitting risk); a very large λ forces the weights towards zero and causes underfitting; the best λ is chosen using validation data. The gradient update becomes $W \leftarrow W - \eta(\partial L/\partial W + 2\lambda W)$.

d) Dropout

During training, dropout randomly sets a fraction p of neuron outputs to zero in each iteration (e.g., p = 0.3–0.5). The network cannot depend on specific neurons or co-adapted features, so it learns redundant, robust representations; it behaves like averaging many thinner sub-networks. At test time all neurons are used with scaled outputs. This reduces the training–validation gap.

TASK 9 – Convolution Operation and Feature Extraction

a) Why convolutional layers suit image data

Images have local spatial structure: spots, edges and textures are formed by neighbouring pixels. A convolutional layer uses small filters that slide over the image (local connectivity) and reuse the same weights at all positions (parameter sharing), so it detects a pattern wherever it appears (translation equivariance) with far fewer parameters than a fully connected layer. This makes it suitable for detecting leaf spots, lesion edges and texture changes.

b) Convolution example Consider a 4 × 4 image (left half bright = 10, right half dark = 0) and a 3 × 3 vertical-edge filter, stride 1, no padding:

Image					Filter			
10	10	0	0		1	0	-1	
10	10	0	0		1	0	-1	
10	10	0	0		1	0	-1	
10	10	0	0					

Output(0,0) = 3×[(10×1)+(10×0)+(0×−1)] = 30 Output(0,1) = 3×[(10×1)+(0×0)+(0×−1)] = 30

Similarly the bottom positions give 30, so the feature map is [[30, 30], [30, 30]] (size = (4 − 3)/1 + 1 = 2). Interpretation: the high positive response shows that the filter has found a vertical edge (bright-to-dark transition); a uniform region would give 0. In leaf images, such filters respond to lesion boundaries (see Graph 4).

c) Roles of filters, stride and padding

• Filters (kernels): small learnable weight matrices; each filter detects one feature (edge, spot, texture) and produces one feature map.

• Stride: the number of pixels the filter moves each step; a larger stride gives a smaller feature map and less computation but loses detail.

• Padding: adding zeros around the border; 'same' padding preserves the spatial size and keeps border information, 'valid' (no padding) shrinks the map.

Output size = (W − F + 2P)/S + 1.

TASK 10 – CNN Architecture and Pooling

a) Function of each component

Component	**Function**
Input Image	Leaf image (e.g., 64 × 64 × 3 RGB) given as a pixel array.
Convolution	Applies learnable filters to extract local features such as edges, spots, and textures.
ReLU	Adds non-linearity by setting negative values to 0, helping the network learn complex patterns and reducing the vanishing gradient problem.
Pooling	Down-samples feature maps, reduces size and computation, and provides small translation invariance.

2nd Conv + ReLU + Pooling	Combines earlier features into more complex patterns, such as lesion shapes and leaf-vein patterns.
Fully Connected Layer	Flattens the final feature maps and combines all features for classification.
Output	Softmax/Sigmoid gives probabilities for healthy vs. diseased leaves.

b) How pooling reduces dimensions

Max-pooling with a 2 × 2 window and stride 2 takes the maximum value of each non-overlapping block, reducing height and width by half (a 4 × 4 map becomes 2 × 2, i.e., 75% fewer values):

1	3	2	4			6	8
5	6	1	2	→ 2×2 max-pool →		4	9
7	2	9	0				
1	8	3	5				

Output size = $\lfloor (W - F)/S \rfloor + 1$. Average pooling takes the mean instead. Example: 64×64×16 → pooling → 32×32×16.

c) Effect of filter size, stride and number of filters

• Filter size: small filters (3 × 3) capture fine details with few parameters; large filters see wider context but have more parameters and reduce the output size more.

• Stride: larger stride → smaller feature maps and lower computation, but may skip fine features.

• Number of filters: determines the depth (channels) of the output; more filters learn more features but increase parameters and memory.

Example: input 64×64×3 → Conv(3×3, 16 filters, stride 1, padding 1) → 64×64×16 → Pool 2×2 → 32×32×16 → Conv(3×3, 32 filters, padding 1) → 32×32×32 → Pool 2×2 → 16×16×32 = 8,192 values → FC → output.

d) Hierarchical feature learning

The first convolutional layer learns low-level features such as edges, colour changes and simple textures. Deeper layers combine them into mid-level patterns (spots, vein structures, lesion shapes) and finally high-level concepts (a specific disease pattern). Pooling increases the receptive field of each neuron, so deeper neurons "see" a larger part of the leaf.

TASK 11 – Comparative Evaluation of Feed-Forward and CNN Models

a) Why Model A suits structured numerical data Weather and soil data are a fixed-length vector of independent numerical features (rainfall, temperature, soil moisture, fertilizer) without spatial structure. A feed-forward network connects all inputs to all neurons, so it can learn arbitrary non-linear interactions between features (e.g., rainfall × fertilizer) with a small number of parameters.

b) Why Model B suits images Leaf images have spatial correlation and local patterns. A CNN exploits local connectivity, shared filters and pooling to

extract hierarchical visual features with relatively few parameters, and it is robust to shifts of the pattern within the image.

c)Comparison

Aspect	**Model A: Feed-forward**	**Model B: CNN**
Input Representation	1-D vector of numerical features (e.g., 4 values).	3-D tensor (height × width × channels), e.g., 224 × 224 × 3.
Feature Extraction	Learned by dense layers from given features; no spatial awareness.	Automatic hierarchical extraction using filters (edges → textures → shapes).
Parameter Organization	Full weight matrices between layers; each weight is used once.	Small shared filters reused over the image; far fewer parameters.
Computational Requirement	Low; trains on a CPU.	High; needs GPU, memory, and larger data.

d)Recommendation

• Crop-yield prediction: Model A (feed-forward regression network) – the data are structured and numerical, and it is simple, fast and interpretable.

• Leaf-disease classification: Model B (CNN) – the data are images; a feed-forward network on raw pixels would be huge (224×224×3 = 150,528 inputs) and ignore spatial structure.

TASK 12 – Design Challenge:

Optimize and Redesign the Deep Learning System Decision: REDESIGN – keep a (modified, regularized) feed-forward network for numerical data and add a separate CNN pipeline for leaf images. A. Impact Analysis

a) Effect of larger data. Going from 10,000 to 500,000 records is a 50× increase: more memory, longer epochs and the need for mini-batch training, an efficient data pipeline (loading, caching, prefetching) and possibly GPU acceleration. The larger dataset allows a deeper model and reduces overfitting if the data are representative. The 100,000 leaf images (e.g., 224×224×3 ≈ 150 KB each as uint8, ≈ 15 GB raw) need much more storage, memory and computation, and a different architecture that can process spatial data. Training with data-parallel mini-batches (e.g., 64–256) and mixed-precision is advisable.

b) Limitations of feed-forward networks on images.

(i) Flattening destroys spatial structure;

(ii) too many parameters – a 224×224×3 image has 150,528 inputs, so one hidden layer of 1,000 neurons already needs ≈ 150.5 million weights (Graph 5);

(iii) no parameter sharing, so the same lesion at a different position must be learned again;

(iv) no translation invariance;

(v) high overfitting risk and heavy computation.

B. Evaluation of the Existing Design

a) High training accuracy with lower validation performance indicates overfitting (high variance): the model memorizes the 10,000 records instead of generalizing. The long training time shows computational inefficiency (poor optimizer/learning-rate settings, no batch normalization, or an oversized model).

b) Simply increasing depth and neurons is not necessarily the best solution: it increases capacity, which usually worsens overfitting, and it increases parameters, training time and the risk of vanishing/exploding gradients. Better options are more data (now available), regularization, a better optimizer and learning-rate schedule, early stopping, and the right architecture for each data type (a CNN for images).

C. Adaptation or Redesign

a) Proposed architectures

System	Architecture
Numerical crop data (500,000 records)	Input (4 or more scaled features) → Dense 64 + BatchNorm + ReLU → Dropout 0.2 → Dense 32 + BatchNorm + ReLU → Dropout 0.2 → Dense 1 (linear). Loss: MSE (or Huber).
Leaf images (100,000 images)	Input 224×224×3 → [Conv 3×3 (32) + BN + ReLU → MaxPool] → [Conv 3×3 (64) + BN + ReLU → MaxPool] → [Conv 3×3 (128) + BN + ReLU → MaxPool] → Global average pooling → Dropout 0.5 → Dense → Softmax (healthy/diseased). Loss: cross-entropy. Better still: transfer learning from a pretrained ResNet-18/50 or EfficientNet-B0, fine-tuned on the leaf images.

b) Optimization and regularization

• Optimizer: Adam (lr = 1e-3, then lower for fine-tuning) as the default for fast convergence; SGD with momentum (0.9) and a schedule can give slightly better final generalization for CNNs.
• Learning rate: warm-up followed by cosine decay, or ReduceLROnPlateau (Graph 6).
• Regularization: L2 weight decay ($\lambda \approx$ 1e-4), dropout (0.2–0.5), early stopping, data augmentation (images).
• Batch normalization after convolution/dense layers for faster and more stable training.
• Activations: ReLU/Leaky ReLU in hidden layers, linear output for yield regression, softmax (or sigmoid) for classification; He initialization.

D. Evidence-Based Defence

a) Experiment design. Use the same data, same train/validation/test split (e.g., 70/15/15) and same preprocessing for the original and the redesigned systems; fit scalers on the training data only. Run repeated trials with different seeds.

Measure	**Numerical Model (Regression)**	**Image Model (Classification)**
Loss	Training and validation MSE	Training and validation cross-entropy
Performance	MAE, RMSE, R^2	Accuracy, Precision, Recall, F1-score, ROC-AUC, Confusion Matrix
Generalization	Gap between training and validation error	Gap between training and validation accuracy
Efficiency	Training time per epoch, total training time, memory usage	Training time per epoch, total training time, GPU memory, inference time

c) Justification.

The expected evidence: the redesigned system shows lower validation loss, a smaller training–validation gap, higher validation accuracy (images) and lower RMSE (yield), and shorter or acceptable training time per epoch because of mini-batches, batch normalization and Adam. If the original model's validation error stays higher and its gap remains large, then it should not be retained as-is: the numerical model is modified (regularization + better optimization), and the image task needs a new CNN – hence an overall redesign.

Additional design considerations

1. Optimizer comparison

Optimizer	**Idea**	**Advantages**	**Limitations**
SGD	Updates using the gradient of the full dataset.	Simple, good final generalization.	Slow; costly for 500,000 records.
Mini-batch SGD	Calculates gradients from small batches (32–256).	Efficient, GPU-friendly; some noise helps escape poor minima.	Noisy; learning rate must be tuned.

Momentum	Adds a velocity term based on past gradients.	Faster convergence; reduces oscillations in ravines.	Extra hyperparameter; may overshoot.
RMSProp	Divides the step by the running average of squared gradients.	Adaptive per-parameter learning rates; handles varying gradient scales.	No momentum; sensitive to decay rate and learning rate.
Adam	Combines Momentum and RMSProp with bias correction.	Fast convergence; robust default choice.	May generalize slightly worse than tuned SGD; requires more memory.

Learning-rate schedule:3-epoch warm-up from 1e-4 to 1e-3, then cosine decay to 1e-5 over about 50 epochs (alternatively reduce by a factor of 10 when validation loss stops improving for 3 epochs). A smaller rate (about 1e-4) is used when fine-tuning pretrained layers.

Label-preserving image augmentation (training set only): horizontal/vertical flips, rotation (±20–30°), small zoom or random crop (80–100%, ensuring that lesions are not removed), slight CSA47 – Deep Learning | Assignment 2 | Page 9 translation, small brightness/contrast changes and mild Gaussian noise/blur. Avoid strong hue/colour changes (colour carries disease information), heavy cropping or distortions that could change the diagnosis. Validation and test images are not augmented.

Comparison of image models (224×224 leaf images, limited compute):

Model	**Input Size**	**Parameters**	**Depth / Key Idea**	**Suitability**
LeNet-5	32 × 32 (Grayscale)	≈ 60 K	2 convolutional layers + 3 fully connected layers	Very lightweight but too small for detailed leaf textures at 224 × 224.

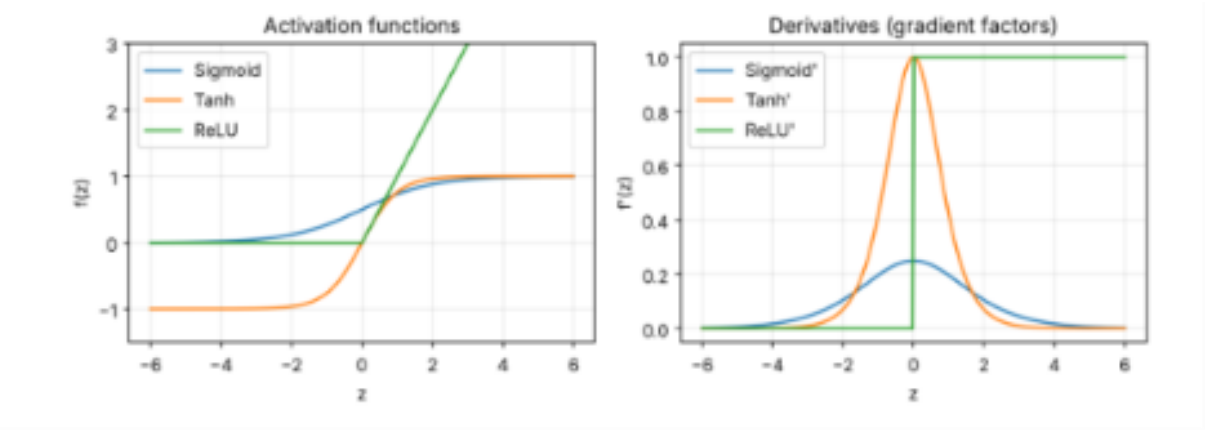
AlexNet	227 × 227	≈ 61 M	5 convolutional layers + 3 fully connected layers; ReLU and Dropout	Large fully connected layers; computationally heavy and outdated.
ResNet-18/50	224 × 224	11.7 M / 25.6 M	Residual (skip) connections	Strong accuracy, easy to train, and good for transfer learning.
EfficientNet-B0	224 × 224	≈ 5.3 M	Compound scaling of depth, width, and resolution	Good accuracy per computation; ideal for a limited computational budget.

Recommendation: fine-tune a pretrained EfficientNet-B0 (or ResNet-18 if simpler/faster) with Adam + cosine learning-rate decay, dropout, L2 and augmentation; this provides high accuracy with a moderate computational budget. A custom small CNN can be used as the baseline.

GRAPHICAL REPRESENTATIONS

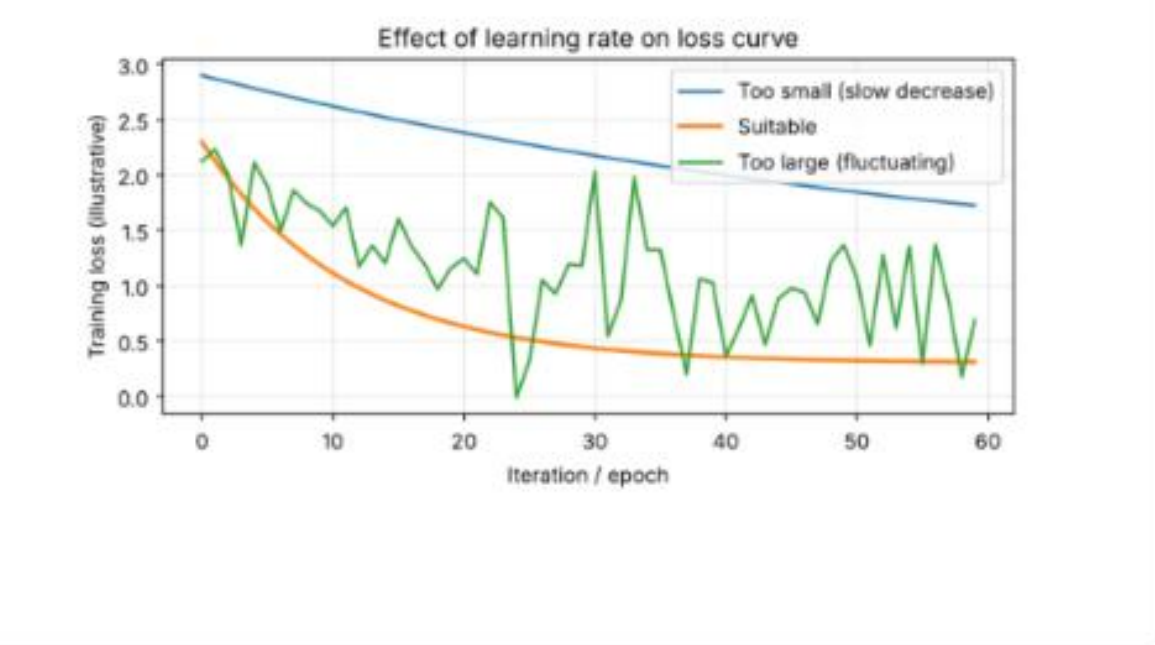
Graph 1 – Activation Functions and Their Derivatives (Task 6)

Sigmoid has a maximum derivative of 0.25 and saturates for large |z|; tanh has a maximum derivative of 1; ReLU has derivative 1 for $z > 0$. This explains why sigmoid leads to vanishing gradients in deep networks.



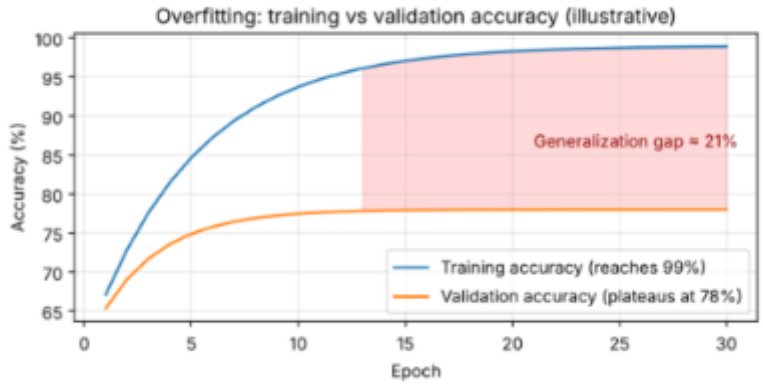
Graph 2 – Effect of Learning Rate on the Loss Curve (Tasks 4 and 7)

This is a conceptual illustration, not measured data. A small learning rate decreases the loss slowly, a suitable rate converges smoothly, and a large rate gives fluctuation.



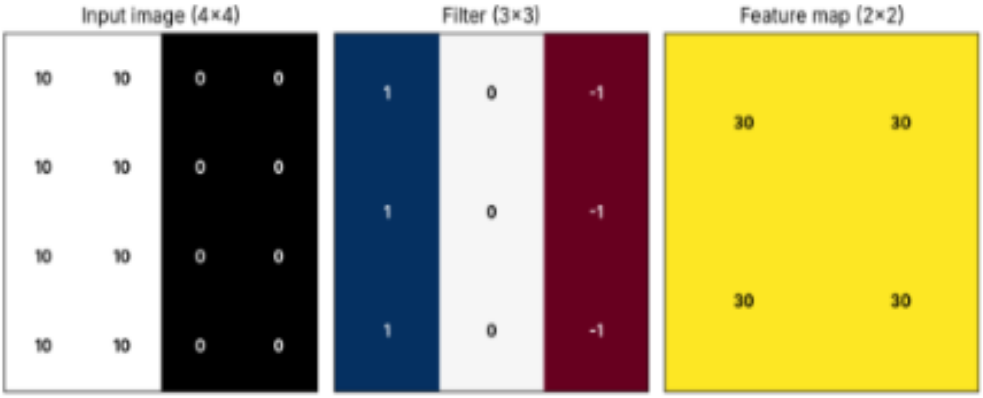
Graph 3 – Overfitting: Training vs Validation Accuracy (Task 8)

Illustrative curves for training accuracy of 99% and validation accuracy of 78%. The widening gap indicates overfitting.



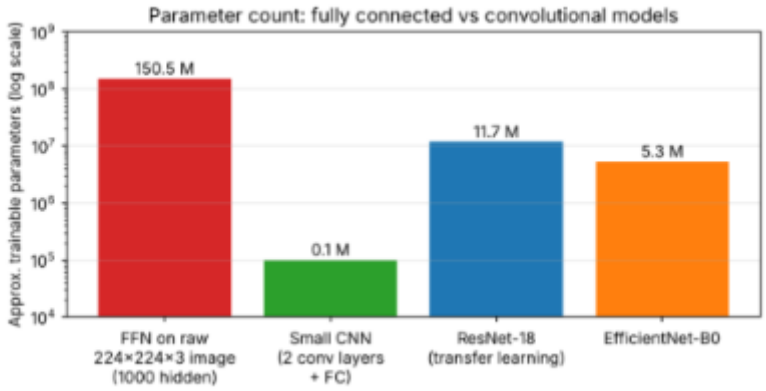
Graph 4 – Convolution Example (Task 9)

The 3×3 vertical-edge filter applied to the 4×4 image gives a 2×2 feature map with strong responses (30) at the vertical edge.



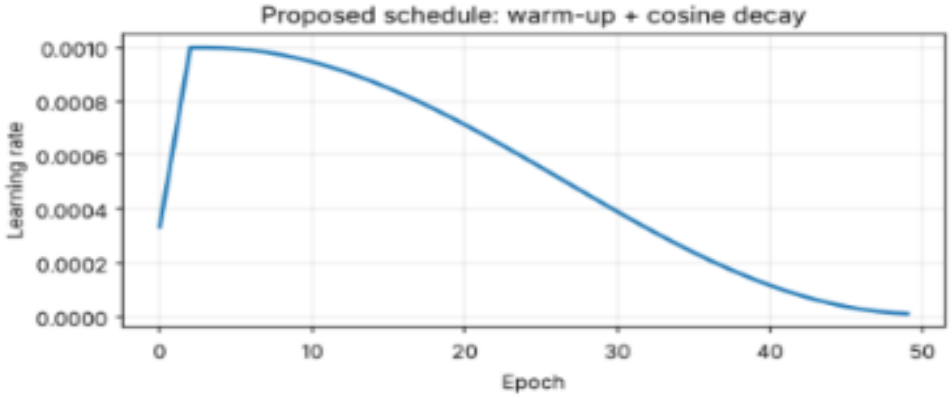
Graph 5 – Parameter Count: Fully Connected vs Convolutional Models (Task 12)

A single dense hidden layer of 1,000 neurons on a 224×224×3 image requires about 150.5 million weights, whereas CNN-based models need far fewer parameters (approximate values).



Graph 6 – Proposed Learning-Rate Schedule (Task 12)

A 3-epoch warm-up to 1e-3 followed by cosine decay to 1e-5 over 50 epochs.



RESULT:

The assignment demonstrates the design, training and analysis of deep feed-forward and convolutional networks for smart agriculture. A 4 → 8 → 4 → 1 feed-forward network (81 trainable parameters) was designed for crop-yield prediction, with weight shapes 8×4, 4×8 and 1×4 and bias shapes 8×1, 4×1 and 1×1. A neuron with weighted sum 0.68 gave a ReLU output of 0.68. For a prediction of 42 against an actual yield of 50, the squared-error loss was 64. A gradient-descent update with w = 0.5, gradient 0.8 and learning rate 0.1 gave a new weight of 0.42. Back-propagation, the chain rule, activation functions (vanishing/exploding gradients), learning-rate effects, regularization (L1, L2, dropout), convolution (a 4×4 image with a 3×3 filter giving a 2×2 feature map of 30s), pooling and CNN architecture were explained. Model A (feed-forward) is recommended for numerical weather and soil data, and Model B (CNN, preferably fine-tuned EfficientNet-B0 or ResNet) for leaf images. For the larger data (500,000 records and 100,000 images) the existing system should be redesigned using separate numerical and image models, Adam/SGD with a learning-rate schedule, batch normalization, dropout, L2 regularization and label-preserving augmentation, and evaluated by loss, validation metrics and training time.